



SmartPot

DOCUMENTACIÓN DEL COMPONENTE

SmartPot-IoT

El firmware de un cultivo real

MicroPython en un ESP32, físico o simulado en Wokwi: sensores, pantalla, actuadores y contrato MQTT v1 con TLS, con su circuito, su ciclo principal, su configuración y sus pruebas.



DOCUMENTO
SmartPot-IoT

VERSIÓN
1.1 · octubre 2026

EQUIPO
SmartPotTech

PROYECTO
smartpot.app



DOCUMENTO

Ficha del documento

CAMPO	VALOR
Proyecto	SmartPot · smartpot.app
Componente	SmartPot-IoT
Versión	1.1 · octubre 2026
Alcance	Circuito, módulos del firmware, ciclo principal, comandos, conexión segura, configuración y pruebas
Documentación de la plataforma	Documentación técnica , recorrido del proyecto , ciclo de vida y diagramas generales
Mantenimiento	Se genera desde docs/ de este repositorio con las herramientas de .github/docs/tools ; se actualiza con cada cambio del componente

CONTENIDO

Contenido

1. Propósito
2. Arquitectura del componente
3. Ciclo principal
4. Comandos
5. Conexión segura
6. Configuración
7. Pruebas
8. Puesta en marcha



PARTE I

El componente

SECCIÓN 01

Propósito

En palabras simples

SmartPot-IoT es el programa que corre en el dispositivo de un **cultivo real**: un ESP32 con sus sensores, sus actuadores y una pantalla. Mide, muestra, publica las lecturas en el broker de SmartPot y obedece las órdenes que llegan, confirmando cada una. No decide nada por su cuenta: la API y el asistente deciden. El mismo firmware corre en una placa física o en [Wokwi](#), y en los dos casos el cultivo es real para la plataforma; los cultivos virtuales, en cambio, los simula SmartPot-DataGenerator.

SECCIÓN 02

Arquitectura del componente

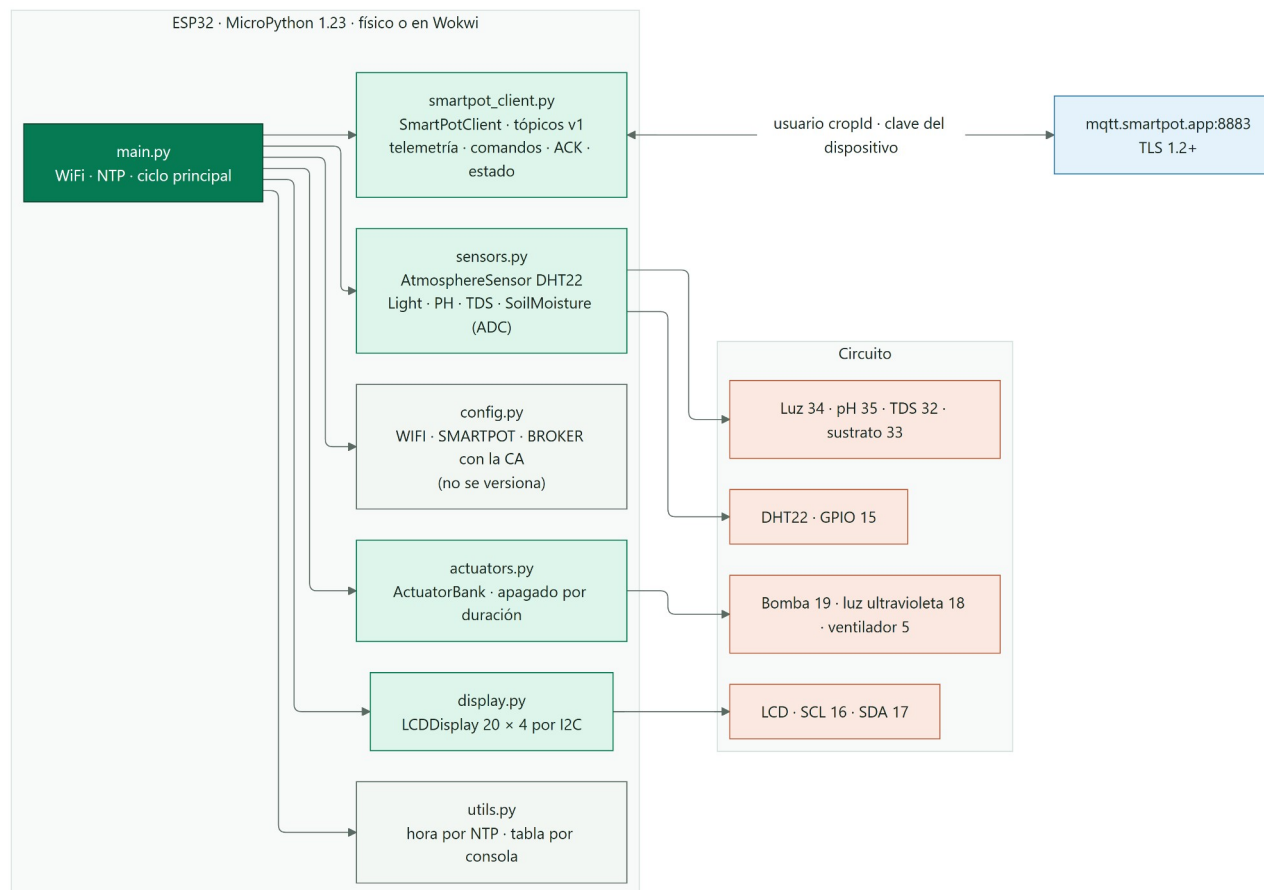


Figura 1 · SmartPot-IoT por dentro



COMPONENTE	PIN ESP32	ESCALA ENVIADA
DHT22 (temperatura y humedad del aire)	GPIO 15	°C y %
Sensor de luz (potenciómetro en Wokwi)	GPIO 34	0-2000 lux
Sensor de pH	GPIO 35	0-14
Sensor de TDS	GPIO 32	0-3000 ppm
Humedad del sustrato	GPIO 33	0-100 %
Bomba de agua	GPIO 19	WATER_PUMP
Luz ultravioleta	GPIO 18	UV_LIGHT
Ventilador	GPIO 5	FAN
LCD 20×4 I2C	SCL 16 · SDA 17	—



PARTE II

Funcionamiento

SECCIÓN 03

Ciclo principal

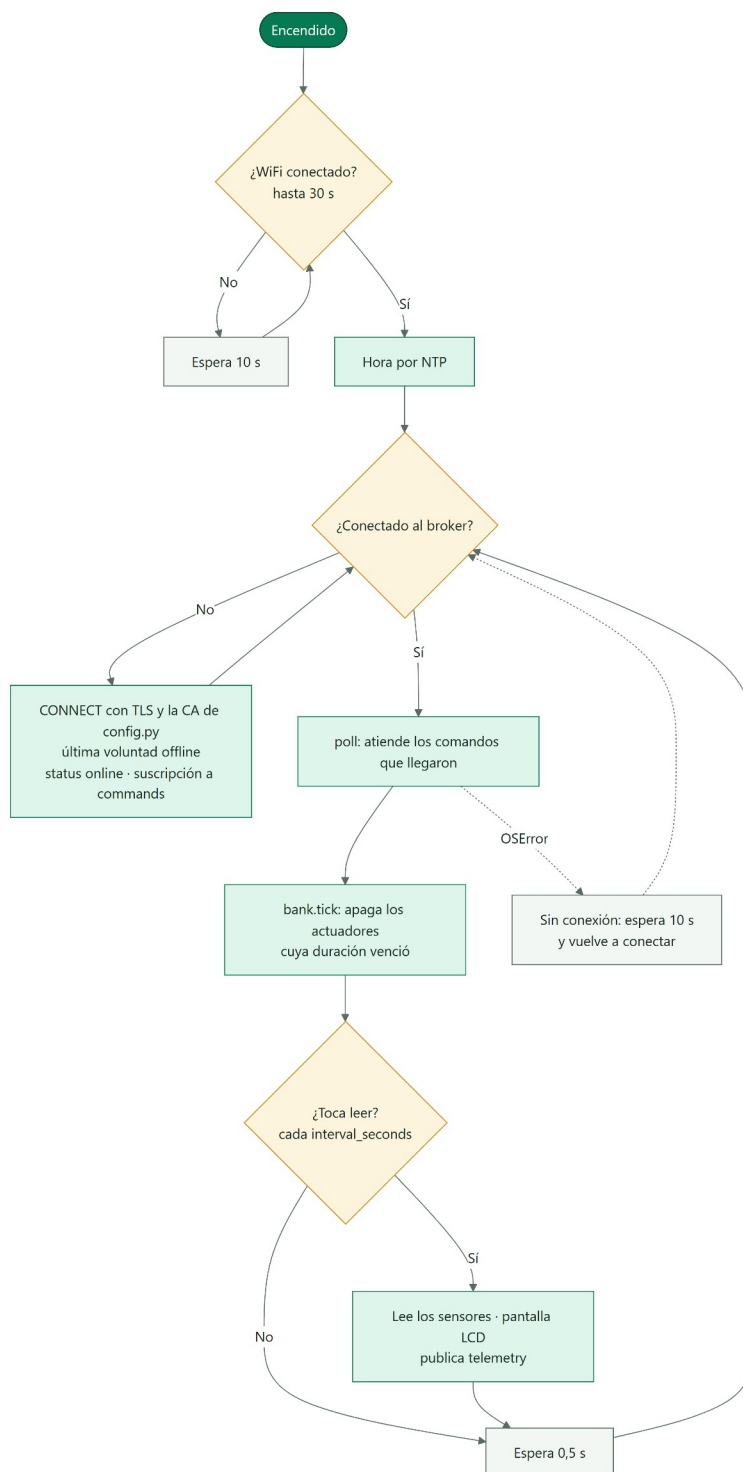


Figura 2 · Ciclo principal del firmware

SECCIÓN 04

Comandos

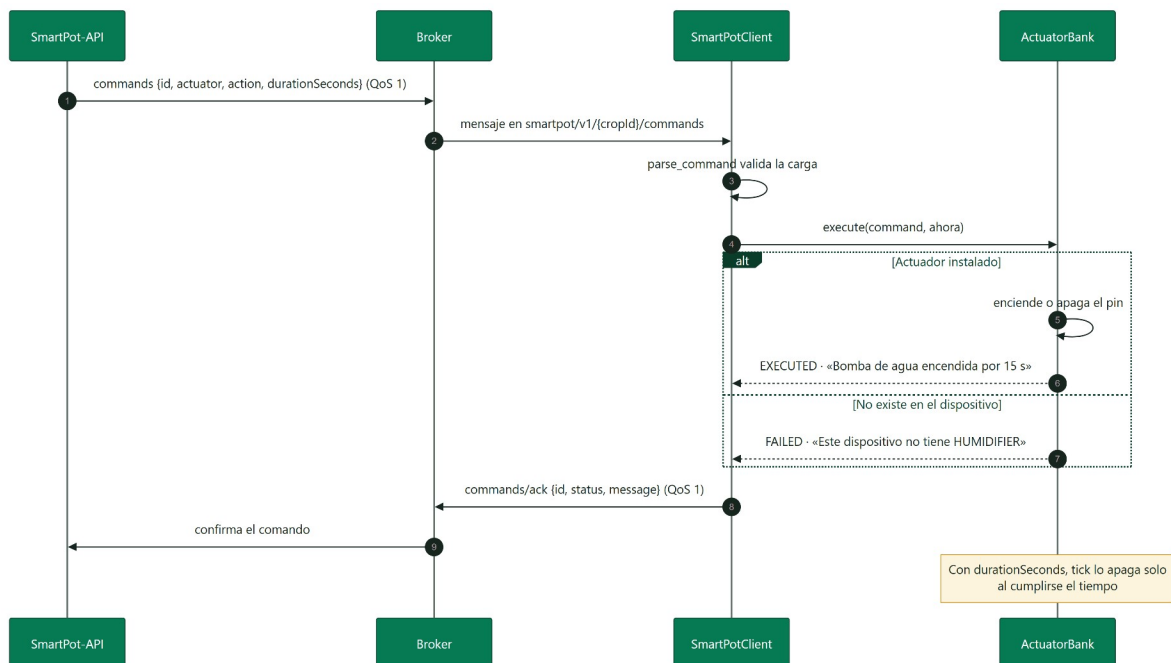


Figura 3 · Cómo atiende una orden

SECCIÓN 05

Conexión segura

PARÁMETRO	VALOR
Broker	mqtt.smartpot.app:8883, MQTT sobre TLS 1.2 o superior
Certificado	Firmado por la CA propia de SmartPot; el firmware lo verifica con la CA pública que trae config.py en BROKER["CA_CRT"]
Usuario y clave	El id del cultivo y la clave del dispositivo, que la PWA muestra una sola vez al crear el cultivo real o al rotarla
Client id	smartpot-<cropId>
Estado	online retenido al conectar y offline como última voluntad

WOKWI

Deja tu copia del proyecto como privada: quien vea config.py puede publicar como tu cultivo. Si se filtra, genera una nueva clave desde la pestaña Dispositivo.

PARTE III

Operación

SECCIÓN 06

Configuración

La PWA genera `config.py` en su guía de conexión, con la red WiFi (la tuya o `Wokwi-GUEST`), el bloque SMARTPOT del cultivo y el bloque BROKER con la CA del broker. `config.py` no se versiona; `config.example.py` es la plantilla.

CLAVE	USO
<code>WIFI.ssid, WIFI.password</code>	Red a la que se conecta el ESP32
<code>SMARTPOT.crop_id, device_key</code>	Cuenta MQTT del cultivo real
<code>SMARTPOT.host, port, tls</code>	Broker y uso de TLS
<code>BROKER.CA_CRT</code>	CA pública de SmartPot en PEM; con ella el firmware verifica el certificado del broker
<code>SMARTPOT.interval_seconds</code>	Segundos entre lecturas (30 por defecto)

SECCIÓN 07

Pruebas

`uv run ruff check .` y `uv run pytest`: 15 pruebas con CPython y módulos de MicroPython simulados sobre el contrato MQTT, los comandos y su ACK (con la duración en segundos, minutos u horas), el apagado por tiempo de los actuadores, la escala de los sensores, la verificación del broker con la CA de la plantilla y el respaldo de TLS en versiones anteriores de MicroPython.



SECCIÓN 08

Puesta en marcha

DÓNDE	PASOS
Placa física	Arma el circuito, graba MicroPython 1.23, copia <code>fs/</code> con <code>mpremote</code> , agrega tu <code>config.py</code> (ya trae la CA) y enciende
Wokwi en el navegador	Abre el proyecto, pega tu <code>config.py</code> y ejecuta; la red Wokwi-GUEST tiene salida a internet
Wokwi local	<code>uv sync</code> , inicia la simulación con <code>wokwi.toml</code> y ejecuta <code>uv run python start.py</code>